

# 01 — Modélisation POO & Première API REST avec FastAPI

- **Séances couvertes :** Séance 1 & Séance 2
  - **Objectifs de la journée :**
    - Cadrer l’univers et le scénario de votre groupe pour le projet fil rouge.
    - Modéliser la structure de votre jeu en POO pure (dataclasses, héritage, polymorphisme, typage).
    - Déployer votre premier serveur FastAPI avec auto-documentation OpenAPI / Swagger UI.
    - Valider strictement les requêtes des Joueurs grâce aux schémas **Pydantic**.
- 

## Brief Client : Mission “DigitalEscape Studio”

*“Bienvenue dans l’équipe de développement ! DigitalEscape Studio vous confie la réalisation du moteur backend de son tout nouveau jeu d’Escape Game interactif. En tant qu’équipe de développeurs / Game Masters, vous avez carte blanche pour choisir la thématique de votre univers (Cyberpunk, Manoir Hanté, Station Spatiale, Braquage, etc.). Votre première journée consiste à poser l’architecture Orientée Objet du domaine et à publier la version Alpha de votre API d’exploration.”*

---

## Partie 1 : Modélisation POO du Domaine Métier (app/domain/)

### Notions Théoriques Clés

- **Type Hints :** `def inspect_item(item_id: str) -> dict:`
- **Dataclasses :** `@dataclass` pour simplifier les structures de données.
- **Héritage & Polymorphisme :** Classe de base `Puzzle` déclinée en `CodePuzzle` et `HashPuzzle`.

### Travail à Réaliser en Équipe

#### Étape 1.1 : Cadrage du Scénario

Avec votre groupe (2 à 3 personnes), définissez brièvement votre univers (3 à 4 salles minimum) et complétez la fiche de cadrage : - **Titre de votre jeu :** (ex: *Operation: Neon Cyberpunk / Le Secret du Château Blackwood*) - **Thématique :** (ex: *Infiltration, Horreur, Sci-Fi...*) - **Nom du Game Master :** (ex: *IA-Sentinel / Le Gardien*)

## Étape 1.2 : Modélisation des Classes métier dans app/domain/

Créez les fichiers Python dans app/domain/ :

### 1. **GameElement** (Classe mère) :

- Attributs : `id: str`, `name: str`, `description: str`.
- Méthode : `to_dict()` -> dict.

### 2. **Item** et **Door** (Héritent de **GameElement**) :

- **Door** possède les attributs `is_locked: bool` et `required_item_id: Optional[str]`.

### 3. **Puzzle** (Classe abstraite / Polymorphe) :

- Méthode abstraite : `check_solution(answer: str) -> bool`.
- **Déclinaison 1 : CodePuzzle**
  - Compare directement la réponse texte ou le code numérique saisi avec un code secret stocké (ex: `answer == "1234"`).
- **Déclinaison 2 : HashPuzzle (Focus Sécurité / Cryptographie)**
  - **Principe** : En sécurité, on ne stocke jamais un mot de passe ou une clé secrète en texte clair ! On stocke son *empreinte cryptographique* (son Hash SHA-256 ou MD5).
  - **Attribut spécifique** : `expected_hash: str` (ex: `"5e884898da28047151d0e56f8dc6292773603"` pour le mot "password").
  - **Fonctionnement de `check_solution(answer: str) -> bool`** : Lorsque le joueur soumet un mot de passe `answer`, la méthode calcule l'empreinte SHA-256 de cette tentative et la compare au hash attendu `expected_hash`.

### 4. **Room** (Héritage à définir) :

- Attributs : `id`, `name`, `description`, `items: list[Item]`, `doors: list[Door]`, `puzzles: list[Puzzle]`.

## Étape 1.3 : Tests et diagrammes

Comment feriez vous pour tester vos classes? Mettez en oeuvre votre solution. Faire un ou plusieurs diagrammes de classe avec mermaid

## Partie 2 : Première API REST & Validation Pydantic (app/routers/)

### Notions Théoriques Clés

- **FastAPI** : Framework ASGI ultra-rapide basé sur les Type Hints.
- **Pydantic (BaseModel)** : Validation automatique des données entrantes/sortantes et génération du contrat OpenAPI JSON.
- **Swagger UI** : Interface interactive accessible sur /docs.

### Travail à Réaliser en Équipe

#### Étape 2.1 : Initialisation de FastAPI (app/main.py)

Créez l'application FastAPI principale et exposez l'endpoint de contrôle de santé :

- GET /health : Retourne {"status": "online", "game\_title": "Votre Titre", "engine\_version": "1.0.0"}.

Exemples d'exploration des salles (GET /salles et GET salle/1)

#### Étape 2.2 : Un escape game se joue à plusieurs

En utilisant ce qui a été vu dans le cours, donner un exemple de la structure pour les joueurs et la possibilité de lister les joueurs et en ajouter... Puis tester tous les types de routes associées aux joueurs.

#### Étape 2.3 : Schemas Pydantic (app/schemas/)

Définissez les modèles de validation d'API par rapport aux classes que vous avez définies:

```
# app/schemas/puzzle.py
from pydantic import BaseModel, Field, field_validator

class PuzzleSubmission(BaseModel):
    puzzle_id: str = Field(..., description="Identifiant unique du puzzle")
    attempt_code: str = Field(..., description="Tentative du joueur (texte, code ou mot de p")
    player_id: str = Field(..., description="Identifiant du joueur")

    @field_validator('attempt_code')
    def code_must_not_be_empty(cls, v):
        # Le code soumis ne peut pas être vide
        # A compléter avec la levée d'une exception ValueError dans le cas contraire
```

## Étape 2.4 : Endpoints REST d'exploration Joueur

Déclarez les routes dans `app/routers/player_game.py` : - `GET /rooms` : Retourne la liste des salles découvertes par le joueur. - `GET /rooms/{room_id}` : Retourne le détail d'une salle spécifique et ses objets interactifs. - `POST /puzzles/submit` : Prend en corps de requête le JSON `PuzzleSubmission` et vérifie si la réponse est correcte.

---

## Checklist de fin de TP

À la fin de ce TP, votre groupe doit pouvoir faire la démonstration suivante au formateur :

- ☐ L'application FastAPI se lance sans erreur via Uvicorn (`uvicorn app.main:app --reload`).
- ☐ L'interface **Swagger UI** est accessible sur `http://127.0.0.1:8000/docs`.
- ☐ Il est possible d'explorer au moins 2 salles de votre scénario via `GET /rooms/{room_id}`.
- ☐ La classe `HashPuzzle` fonctionne et vérifie correctement les empreintes SHA-256.
- ☐ Une tentative de soumission d'un code vide sur `POST /puzzles/submit` déclenche une erreur **422 Unprocessable Entity** automatique de Pydantic.
- ☐ La soumission du bon code sur un puzzle renvoie `{"success": true, "message": "Porte déverrouillée !"}`.